- Guía de Desarrollo Modular (Versión 2): Proyecto de Registro y Dashboard
  - 1. Configuración Inicial
    - 1.1. Crear el Proyecto
    - 1.2. Instalar Dependencias
  - 2. Estructura de Carpetas Modular
  - 3. Módulo de Autenticación (Auth)
    - 3.1. Estilos del Módulo (Auth.css)
    - 3.2. Hooks de Lógica
      - useLoginForm.js
      - useRegisterForm.js
    - 3.3. Componentes de Interfaz
      - LoginForm.jsx
      - RegisterForm.jsx
  - 4. Módulo de Dashboard
    - 4.1. Componentes Atómicos
      - Navbar
      - Hero
      - UpdateForm
    - 4.2. Hook de Lógica del Dashboard (useWelcome.js)
    - 4.3. Página Principal (WelcomePage.jsx)
  - 5. Rutas y App
  - 6. Ejecución

Guía de Desarrollo Modular (Versión 2): Proyecto de Registro y Dashboard

Esta guía detalla el proceso para construir una aplicación React escalable utilizando una **arquitectura modular**. Cada funcionalidad principal (Autenticación, Dashboard) se encuentra aislada en su propio módulo con sus componentes, hooks y estilos.

---

1. Configuración Inicial

1.1. Crear el Proyecto

Genera la base del proyecto con `create-react-app`:

```
npx create-react-app proyecto_v2
cd proyecto_v2
```

1.2. Instalar Dependencias

Instala el enrutador para manejar la navegación:

```
npm install react-router-dom@7.7.0
```

---

2. Estructura de Carpetas Modular

La clave de esta versión es la organización. En lugar de carpetas genéricas (`components`, `hooks`), agrupamos por **funcionalidad**.

Tu árbol de archivos debe verse así:

```
src/
├── modules/
│   ├── auth/              (Módulo de Login/Registro)
│   │   ├── components/
│   │   │   ├── Auth.css
│   │   │   ├── LoginForm.jsx
│   │   │   └── RegisterForm.jsx
│   │   └── hooks/
│   │       ├── useLoginForm.js
│   │       └── useRegisterForm.js
│   │
│   └── dashboard/         (Módulo de Usuario)
│       ├── components/
│       │   ├── Footer.css
│       │   ├── Footer.jsx
│       │   ├── Hero.css
│       │   ├── Hero.jsx
│       │   ├── Navbar.css
│       │   ├── Navbar.jsx
│       │   ├── UpdateForm.css
│       │   └── UpdateForm.jsx
│       ├── hooks/
│       │   └── useWelcome.js
│       └── Pages/
```

```
│           └── WelcomePage.jsx
│
├── shared/                (Recursos compartidos)
│   └── assets/
│       └── Logo.webp
│
├── routes/
│   └── AppRoutes.jsx
│
├── App.css
├── App.js
├── index.css
├── index.js
└── ...
```

> **Nota:** Coloca tu logo en `src/shared/assets/Logo.webp`.

---

3. Módulo de Autenticación (Auth)

Este módulo maneja el inicio de sesión y el registro.

3.1. Estilos del Módulo (`Auth.css`)

Archivo: `src/modules/auth/components/Auth.css`

```css
/* Auth.css - Estilos específicos para autenticación */
.login{
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.login-container{
  width: 600px;
  padding: 30px;
  background-color: #fff;
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  margin-left: auto;
  margin-right: auto;
  margin-top: 70px;
}

.login-container h2{ text-align: center; margin-bottom: 20px; }
.login-container form{ display: flex; flex-direction: column; width: 100%; }
.login-container label{ margin-bottom: 10px; }
```

```
.login-container input{ padding: 10px; margin-bottom: 20px; border: 1px solid #ccc;
border-radius: 4px; width: 100%; }
.login-container button{ padding: 10px; background-color: #28a745; color: #fff;
border: none; border-radius: 4px; cursor: pointer; }
.login-container button:hover{ background-color: #218838; }
.login-container .error{ color: red; margin-bottom: 10px; }
```

3.2. Hooks de Lógica

`useLoginForm.js`

Archivo: `src/modules/auth/hooks/useLoginForm.js`

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';

const initialData = { email: '', password: '' };
const initialErrors = { email: '', password: '' };
const initialTouched = { email: false, password: false };

const useLoginForm = () => {
    const [loginData, setLoginData] = useState(initialData);
    const [fieldErrors, setFieldErrors] = useState(initialErrors);
    const [touched, setTouched] = useState(initialTouched);
    const [submitError, setSubmitError] = useState('');
    const navigate = useNavigate();

    const validateField = (field, value) => {
        const trimmedValue = value.trim();
        if (!trimmedValue) return 'Este campo es obligatorio';
        if (field === 'email' && !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(trimmedValue))
{
            return 'Por favor ingrese un email valido';
        }
        return '';
    };

    const validateForm = (data) => ({
        email: validateField('email', data.email),
        password: validateField('password', data.password)
    });

    const hasErrors = (errors) => Object.values(errors).some((error) => error);

    const handleChange = (e) => {
        const { id, value } = e.target;
        setLoginData({ ...loginData, [id]: value });
        if (touched[id]) {
            setFieldErrors(prev => ({ ...prev, [id]: validateField(id, value) }));
        }
        if (submitError) setSubmitError('');
```

```
    };

    const handleBlur = (e) => {
        const { id, value } = e.target;
        setTouched(prev => ({ ...prev, [id]: true }));
        setFieldErrors(prev => ({ ...prev, [id]: validateField(id, value) }));
    };

    const handleSubmit = (e) => {
        e.preventDefault();
        const touchedAll = { email: true, password: true };
        const validation = validateForm(loginData);
        setTouched(touchedAll);
        setFieldErrors(validation);
        setSubmitError('');

        if (hasErrors(validation)) return;

        const email = loginData.email.trim().toLowerCase();
        const password = loginData.password.trim();
        const users = JSON.parse(localStorage.getItem('users')) || [];
        const user = users.find((u) => u.email.toLowerCase() === email &&
u.password === password);

        if (user) {
            localStorage.setItem('currentUser', JSON.stringify(user));
            navigate('/welcome');
            return;
        }
        setSubmitError('Email o contraseña incorrectos');
    };

    return { loginData, fieldErrors, touched, submitError, handleChange,
handleBlur, handleSubmit };
};

export default useLoginForm;
```

useRegisterForm.js

Archivo: src/modules/auth/hooks/useRegisterForm.js

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';

const initialData = { username: '', password: '', email: '', telefono: '',
fechaNacimiento: '' };
const initialErrors = { username: '', password: '', email: '', telefono: '',
fechaNacimiento: '' };
const initialTouched = { username: false, password: false, email: false, telefono:
false, fechaNacimiento: false };

const useRegisterForm = () => {
```

```
    const navigate = useNavigate();
    const [formData, setFormData] = useState(initialData);
    const [fieldErrors, setFieldErrors] = useState(initialErrors);
    const [touched, setTouched] = useState(initialTouched);
    const [submitError, setSubmitError] = useState('');

    const validateField = (field, value) => {
        const trimmedValue = value.trim();
        if (!trimmedValue) return 'Este campo es obligatorio';
        if (field === 'username' && trimmedValue.length < 3) return 'El nombre debe
tener mínimo 3 caracteres';
        if (field === 'email') {
            if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(trimmedValue)) return 'Email
inválido';
            const users = JSON.parse(localStorage.getItem('users')) || [];
            if (users.some(u => u.email.toLowerCase() ===
trimmedValue.toLowerCase())) return 'El email ya está registrado';
        }
        if (field === 'password' && !/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).
{8,}$/.test(trimmedValue)) return 'La clave debe tener 8 caracteres, mayúscula,
minúscula y número';
        if (field === 'telefono' && !/^\d{7,15}$/.test(trimmedValue)) return
'Teléfono inválido (7-15 dígitos)';
        if (field === 'fechaNacimiento') {
            if (new Date(trimmedValue) > new Date()) return 'Fecha inválida';
        }
        return '';
    };

    const validateForm = (data) => {
        const errors = {};
        Object.keys(data).forEach(field => errors[field] = validateField(field,
data[field]));
        return errors;
    };

    const hasErrors = (errors) => Object.values(errors).some(error => error);

    const handleChange = (e) => {
        const { id, value } = e.target;
        setFormData({ ...formData, [id]: value });
        if (touched[id]) setFieldErrors(prev => ({ ...prev, [id]: validateField(id,
value) }));
    };

    const handleBlur = (e) => {
        const { id, value } = e.target;
        setTouched(prev => ({ ...prev, [id]: true }));
        setFieldErrors(prev => ({ ...prev, [id]: validateField(id, value) }));
    };

    const handleSubmit = (e) => {
        e.preventDefault();
        const touchedAll = { username: true, password: true, email: true, telefono:
true, fechaNacimiento: true };
        const validation = validateForm(formData);
        setTouched(touchedAll);
```

```
        setFieldErrors(validation);

        if (hasErrors(validation)) {
            setSubmitError('Corrija los errores');
            return;
        }

        const newUser = { id: Date.now(), ...formData, email:
formData.email.toLowerCase() };
        const users = JSON.parse(localStorage.getItem('users')) || [];
        users.push(newUser);
        localStorage.setItem('users', JSON.stringify(users));

        alert('Registro exitoso');
        navigate('/login');
    };

    return { formData, fieldErrors, touched, submitError, handleChange, handleBlur,
handleSubmit };
};

export default useRegisterForm;
```

3.3. Componentes de Interfaz

`LoginForm.jsx`

Archivo: `src/modules/auth/components/LoginForm.jsx`

```
import React from 'react';
import useLoginForm from '../hooks/useLoginForm';
import './Auth.css';
import Logo from '../../../shared/assets/Logo.webp';

const LoginForm = () => {
    const { loginData, fieldErrors, touched, submitError, handleChange, handleBlur,
handleSubmit } = useLoginForm();

    return (
        <div className="login-container">
            <h2>Login <img src={Logo} alt="" width="80" /></h2>
            <form onSubmit={handleSubmit} noValidate>
                {submitError && <div className="error-message" style={{ color:
'red' }}>{submitError}</div>}
                <div>
                    <label htmlFor="email">Email:</label>
                    <input type="email" id="email" required value={loginData.email}
onChange={handleChange} onBlur={handleBlur} />
                    {touched.email && fieldErrors.email && <small style={{ color:
'red' }}>{fieldErrors.email}</small>}
                </div>
```

```
                <div>
                    <label htmlFor="password">Contraseña:</label>
                    <input type="password" id="password" required value=
{loginData.password} onChange={handleChange} onBlur={handleBlur} />
                    {touched.password && fieldErrors.password && <small style={{
color: 'red' }}>{fieldErrors.password}</small>}
                </div>
                <button type="submit">Ingresar</button>
                <p>No tienes cuenta? <a href="/register">Registrarse</a></p>
            </form>
        </div>
    );
};

export default LoginForm;
```

RegisterForm.jsx

Archivo: `src/modules/auth/components/RegisterForm.jsx`

```
import React from 'react';
import useRegisterForm from '../hooks/useRegisterForm';
import Logo from '../../../shared/assets/Logo.webp';
import './Auth.css';

const RegisterForm = () => {
    const { formData, fieldErrors, touched, submitError, handleChange, handleBlur,
handleSubmit } = useRegisterForm();

    return (
        <div className="login-container">
            <form onSubmit={handleSubmit} noValidate>
                <h2>Registro <img src={Logo} alt="" width="80" /></h2>
                {submitError && <div className="error-message" style={{ color:
'red' }}>{submitError}</div>}

                {['username', 'password', 'email', 'telefono',
'fechaNacimiento'].map((field) => (
                    <div key={field}>
                        <label htmlFor={field}>{field.charAt(0).toUpperCase() +
field.slice(1)}:</label>
                        <input
                            type={field === 'password' ? 'password' : field ===
'fechaNacimiento' ? 'date' : 'text'}
                            id={field}
                            value={formData[field]}
                            onChange={handleChange}
                            onBlur={handleBlur}
                        />
                        {touched[field] && fieldErrors[field] && <small style={{
color: 'red' }}>{fieldErrors[field]}</small>}
                    </div>
                ))}
```

```
                <button type="submit">Registrar</button>
            </form>
            <p>Ya tienes cuenta? <a href="/login">Iniciar sesión</a></p>
        </div>
    );
};

export default RegisterForm;
```

---

4. Módulo de Dashboard

4.1. Componentes Atómicos

Navbar

Archivo: `src/modules/dashboard/components/Navbar.css`

```
.navbar { display: flex; justify-content: space-between; align-items: center;
padding: 15px 30px; background-color: #343a40; color: white; }
.navbar-brand { display: flex; align-items: center; gap: 15px; }
.navbar-logo { width: 40px; border-radius: 50%; }
.navbar-title { font-size: 1.2rem; font-weight: bold; }
.navbar-user { display: flex; align-items: center; gap: 20px; }
.navbar-welcome { font-size: 0.9rem; }
.navbar-welcome strong { color: #219937; }
.logout-button { padding: 8px 15px; background-color: #dc3545; color: white;
border: none; border-radius: 4px; cursor: pointer; transition: background 0.3s; }
.logout-button:hover { background-color: #c82333; }
```

Archivo: `src/modules/dashboard/components/Navbar.jsx`

```
import React from 'react';
import Logo from '../../../shared/assets/Logo.webp';
import './Navbar.css';

const Navbar = ({ username, onLogout }) => (
    <nav className="navbar">
        <div className="navbar-brand">
            <img src={Logo} alt="Logo" className="navbar-logo" />
            <span className="navbar-title">Dashboard</span>
        </div>
        <div className="navbar-user">
            <span className="navbar-welcome">Hola, <strong>{username}</strong>
</span>
```

```
            <button onClick={onLogout} className="logout-button">Cerrar
Sesión</button>
        </div>
    </nav>
);
export default Navbar;
```

Hero

Archivo: `src/modules/dashboard/components/Hero.css`

```
.hero-container { padding: 40px 20px; text-align: center; background-color: #fff;
border-radius: 12px; margin: 40px auto; max-width: 600px; box-shadow: 0 4px 15px
rgba(0,0,0,0.1); }
.hero-title { color: #333; margin-bottom: 10px; }
.hero-title b { color: #469f39; }
.hero-subtitle { color: #666; margin-bottom: 30px; }
.hero-info-card { text-align: left; background-color: #f8f9fa; padding: 20px;
border-radius: 8px; margin-bottom: 20px; border: 1px solid #e9ecef; }
.info-item { margin: 10px 0; }
.update-button { padding: 12px 24px; background-color: #469f39; color: white;
border: none; border-radius: 6px; cursor: pointer; font-size: 1rem; transition:
background 0.3s; }
.update-button:hover { background-color: #099531; }
```

Archivo: `src/modules/dashboard/components/Hero.jsx`

```
import React from 'react';
import './Hero.css';

const Hero = ({ user, onUpdate }) => (
    <div className="hero-container">
        <h1 className="hero-title">Bienvenido, <b>{user.username}!</b></h1>
        <p className="hero-subtitle">Aquí está tu información personal:</p>
        <div className="hero-info-card">
            <p className="info-item"><strong>Email:</strong> {user.email}</p>
            <p className="info-item"><strong>Teléfono:</strong> {user.telefono}</p>
            <p className="info-item"><strong>Fecha de Nacimiento:</strong> {new
Date(user.fechaNacimiento).toLocaleDateString()}</p>
        </div>
        <button onClick={onUpdate} className="update-button">Actualizar
Información</button>
    </div>
);
export default Hero;
```

UpdateForm

Archivo: `src/modules/dashboard/components/UpdateForm.css`

```
.modal-overlay { position: fixed; top: 0; left: 0; right: 0; bottom: 0; background-
color: rgba(0, 0, 0, 0.5); display: flex; justify-content: center; align-items:
center; z-index: 1000; }
.modal-content { background-color: white; padding: 30px; border-radius: 8px; width:
90%; max-width: 500px; max-height: 90vh; overflow-y: auto; position: relative; box-
shadow: 0 4px 6px rgba(0, 0, 0, 0.1); }
.modal-buttons { display: flex; gap: 10px; margin-top: 20px; }
.modal-buttons button { flex: 1; padding: 10px; border-radius: 4px; border: none;
cursor: pointer; }
.modal-buttons button[type="submit"] { background-color: #28a745; color: white; }
.cancel-button { background-color: #6c757d; color: white; }
```

Archivo: `src/modules/dashboard/components/UpdateForm.jsx`

```
import React from 'react';
import './UpdateForm.css';

const UpdateForm = ({ isOpen, onClose, formData, onChange, onUpdate }) => {
    if (!isOpen) return null;
    return (
        <div className="modal-overlay">
            <div className="modal-content">
                <h2>Actualizar Perfil</h2>
                <form onSubmit={onUpdate}>
                    {/* Campos del formulario */}
                    <div><label>Nombre:</label><input type="text" name="username"
value={formData.username} onChange={onChange} required /></div>
                    <div><label>Teléfono:</label><input type="tel" name="telefono"
value={formData.telefono} onChange={onChange} required /></div>
                    <div><label>Contraseña:</label><input type="password"
name="password" value={formData.password} onChange={onChange} required /></div>
                    <div><label>Fecha:</label><input type="date"
name="fechaNacimiento" value={formData.fechaNacimiento} onChange={onChange}
required /></div>

                    <div className="modal-buttons">
                        <button type="submit">Guardar</button>
                        <button type="button" className="cancel-button" onClick=
{onClose}>Cancelar</button>
                    </div>
                </form>
            </div>
        </div>
    );
};
export default UpdateForm;
```

4.2. Hook de Lógica del Dashboard (`useWelcome.js`)

Archivo: `src/modules/dashboard/hooks/useWelcome.js`

```
import { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';

const useWelcome = () => {
    const navigate = useNavigate();
    const [user, setUser] = useState(null);
    const [isModalOpen, setIsModalOpen] = useState(false);
    const [formData, setFormData] = useState({ username: '', email: '', telefono:
'', password: '', fechaNacimiento: '' });

    useEffect(() => {
        const currentUser = JSON.parse(localStorage.getItem('currentUser'));
        if (!currentUser) navigate('/login');
        else {
            setUser(currentUser);
            setFormData(currentUser);
        }
    }, [navigate]);

    const handleLogout = () => {
        localStorage.removeItem('currentUser');
        navigate('/login');
    };

    const handleInputChange = (e) => {
        const { name, value } = e.target;
        setFormData({ ...formData, [name]: value });
    };

    const handleUpdate = (e) => {
        e.preventDefault();
        const users = JSON.parse(localStorage.getItem('users')) || [];
        const updatedUsers = users.map(u => u.email === user.email ? { ...u,
...formData } : u);

        localStorage.setItem('users', JSON.stringify(updatedUsers));
        localStorage.setItem('currentUser', JSON.stringify(formData));

        setUser(formData);
        setIsModalOpen(false);
        alert('Información actualizada correctamente');
    };

    return { user, isModalOpen, formData, handleLogout, handleInputChange,
handleUpdate, openModal: () => setIsModalOpen(true), closeModal: () =>
setIsModalOpen(false) };
```

```
};
export default useWelcome;
```

4.3. Página Principal (`WelcomePage.jsx`)

Archivo: `src/modules/dashboard/Pages/WelcomePage.jsx`

```
import React from 'react';
import useWelcome from '../hooks/useWelcome';
import UpdateForm from '../components/UpdateForm';
import Navbar from '../components/Navbar';
import Hero from '../components/Hero';
import Footer from '../components/Footer';

const Welcome = () => {
    const { user, isModalOpen, formData, handleLogout, handleInputChange,
handleUpdate, openModal, closeModal } = useWelcome();

    if (!user) return <div>Loading...</div>;

    return (
        <div style={{ display: 'flex', flexDirection: 'column', minHeight: '100vh'
}}>
            <Navbar username={user.username} onLogout={handleLogout} />
            <main style={{ flex: 1, backgroundColor: '#f0f2f5' }}>
                <Hero user={user} onUpdate={openModal} />
            </main>
            <Footer />
            <UpdateForm isOpen={isModalOpen} onClose={closeModal} formData=
{formData} onChange={handleInputChange} onUpdate={handleUpdate} />
        </div>
    );
};
export default Welcome;
```

---

5. Rutas y App

Archivo: `src/routes/AppRoutes.jsx`

```
import React from 'react';
import { Route, Routes } from 'react-router-dom';
import LoginForm from '../modules/auth/components/LoginForm';
import RegisterForm from '../modules/auth/components/RegisterForm';
import WelcomePage from '../modules/dashboard/Pages/WelcomePage';
```

```
const AppRoutes = () => {
    return (
        <Routes>
            <Route path="/" element={<LoginForm />} />
            <Route path="/login" element={<LoginForm />} />
            <Route path="/register" element={<RegisterForm />} />
            <Route path="/welcome" element={<WelcomePage />} />
        </Routes>
    );
};
export default AppRoutes;
```

Archivo: `src/App.js`

```
import './App.css';
import React from 'react';
import AppRoutes from './routes/AppRoutes';

function App() {
  return (
    <main className="Login">
      <AppRoutes />
    </main>
  );
}
export default App;
```

---

6. Ejecución

Para iniciar el proyecto:

```
npm start
```